

Autenticación en API REST con Node.js y Express

0. Instalaciones necesarias para el proyecto

Antes de implementar la autenticación completa en tu API REST con Node.js y Express, necesitás instalar algunos paquetes esenciales usando npm.

Las instalaciones necesarias son:

- Express: Framework para manejar el servidor y las rutas.
- bcryptjs: Para hashear y comparar contraseñas.
- jsonwebtoken: Para generar y verificar tokens JWT.
- cookie-parser: Para leer cookies fácilmente desde las solicitudes del cliente.
- dotenv: Para manejar variables de entorno de forma segura (opcional pero recomendado).

Comando para instalarlos todos juntos:

```
npm install express bcryptjs jsonwebtoken cookie-parser dotenv
```

También necesitás asegurarte de tener Node.js y npm instalados en tu máquina.

Si no los tenés, podés descargarlos desde <https://nodejs.org>

1. ¿Qué es la Autenticación?

La autenticación es el proceso mediante el cual una aplicación confirma que un usuario es quien dice ser.

En una API, es esencial para proteger los datos sensibles y garantizar que sólo los usuarios autorizados accedan a ciertas funcionalidades.

Ejemplo de la vida real: cuando entrás a tu cuenta de Netflix, la plataforma verifica que seas realmente vos.

2. bcrypt: Encriptando Contraseñas

bcrypt es un módulo de Node.js que se utiliza para encriptar (hashear) contraseñas.

Guardar contraseñas en texto plano es peligroso: si una base de datos es hackeada, todos los usuarios estarían en riesgo.

bcrypt usa un proceso llamado 'salting' para proteger aún más la contraseña, haciendo que dos contraseñas

Autenticación en API REST con Node.js y Express

iguales generen hashes diferentes.

Ejemplo real: es como guardar la clave de tu locker en una caja fuerte en lugar de escribirla en la puerta del locker.

Código básico para hashear:

```
const bcrypt = require('bcryptjs');  
  
const hashedPassword = await bcrypt.hash('miPassword123', 10);
```

Código para comparar contraseñas:

```
const match = await bcrypt.compare('miPassword123', hashedPassword);
```

3. JWT: Autenticando Usuarios con Tokens

JWT (JSON Web Token) es una manera de enviar información segura entre un cliente y un servidor.

Es útil en APIs porque evita que tengamos que mantener sesiones pesadas en el servidor.

Un token JWT tiene tres partes: Header (metadatos), Payload (datos) y Signature (firma para validar).

Ejemplo real: es como un pase de entrada a un festival, donde tu nombre y categoría (VIP, General) están codificados en un ticket.

Código básico para crear un token:

```
const jwt = require('jsonwebtoken');  
  
const token = jwt.sign({ id: user.id }, 'secreto', { expiresIn: '1h' });
```

Código para verificar un token:

```
const decoded = jwt.verify(token, 'secreto');
```

4. Cookies: Guardando el Token en el Cliente

Las cookies son pequeños archivos de datos que se guardan en el navegador del cliente.

Son útiles para almacenar el token JWT de forma segura y enviarlo automáticamente en cada solicitud.

Es importante usar opciones como 'HttpOnly' y 'Secure' para proteger las cookies.

Ejemplo real: como dejar tu credencial de acceso en una caja fuerte a la que sólo el guardia puede acceder.

Código para enviar un token en una cookie:

Autenticación en API REST con Node.js y Express

```
res.cookie('token', token, {  
  httpOnly: true,  
  secure: true,  
  sameSite: 'strict'  
});
```

5. Flujo completo en la aplicación

- El usuario se registra -> el servidor hasha la contraseña y la guarda.
- El usuario inicia sesión -> el servidor verifica la contraseña, genera un token JWT y lo envía en una cookie.
- El cliente guarda el token en la cookie.
- Para rutas protegidas, el servidor lee el token de la cookie y valida que sea correcto.

6. Errores comunes a evitar

- Guardar contraseñas en texto plano.
- No usar 'HttpOnly' y 'Secure' en las cookies.
- No verificar si un token ha expirado antes de confiar en él.
- No manejar excepciones en bcrypt o JWT (pueden generar caídas inesperadas).